Voice Interface and Spoken Tool Use

Standalone technical deep-dive dossier

Voice Interface and Spoken Tool Use

Voice control is powerful because it lowers friction, but spoken commands are ambiguous and can trigger tools. The system needed tool routing, muting, fallbacks, destructive-command filtering, and a privacy-forward plan.

- This shows voice as a safety-sensitive command surface, not just a pleasant chat interface.
- Wake-word/macOS app features should be described as planned or partially integrated unless retested.

What I had in mind

Voice was exciting because it made the system feel immediate, but that is also why it is dangerous. If I can speak and the computer acts, then muting, confirmation, shell limits and privacy are part of the product, not accessories.

What I built

- `voice.py` contains Gemini Live config, sample-rate handling, tool declarations, dispatcher, vault tools, deep-think route, shell guard, reconnect/fallback loop, and VoiceBridge experiments. Planning notes define latency targets, privacy red lines, and acceptance criteria.

Mechanism detail

- The voice system is not merely speech-to-text. Spoken commands can trigger tool calls, vault operations, daemon calls and constrained shell execution.
- That makes voice a safety-sensitive surface. Mute behavior, fallbacks, destructive-command filtering, timeouts and privacy boundaries are not optional polish; they are core architecture.
- The later plan around wake-word, local STT, high-quality TTS and barge-in shows awareness of latency, privacy and interruption as engineering constraints.

Architecture

- Python terminal voice interface streams microphone audio into Gemini Live.
- The Live config exposes tools for vault search/read/write, deep thinking through Miguel Core, brain status, context, and constrained shell execution.
- Dangerous shell patterns are blocked and command execution is timeout-limited.
- A later design adds wake-word, local STT, hybrid TTS, barge-in, and privacy boundaries.

Evidence cluster

- Voice source implementing microphone streaming, Gemini Live configuration, tool declarations, dispatcher, vault tools, deep-think route, constrained shell, reconnect, and fallback behavior.
- Voice pipeline notes defining wake-word, local STT, hybrid TTS including local/provider voices, barge-in, latency budget, and privacy red lines.
- VoiceBridge notes separating browser/orb STT work from the local daemon response path.
- Safety-relevant source sections around mute behavior, destructive-command filtering, timeouts, and tool dispatch.

Claim-to-evidence map

The public version should be strong because it is bounded, not because it overclaims.

Claim	Evidence	Boundary
This shows voice as a safety-sensitive command surface, not just a pleasant chat interface.	Voice source implementing microphone streaming, Gemini Live configuration, tool declarations, dispatcher, vault tools, deep-think route, constrained shell, reconnect, and fallback behavior.; Voice pipeline notes defining wake-word, local STT, hybrid TTS including local/provider voices, barge-in, latency budget, and privacy red lines.	Wake-word/macOS app features should be described as planned or partially integrated unless retested.
The work is valuable because it shows mechanism, not surface polish.	Python terminal voice interface streams microphone audio into Gemini Live.; The Live config exposes tools for vault search/read/write, deep thinking through Miguel Core, brain status, context, and constrained shell execution.	Fresh public demos or benchmarks would be the next proof layer.

Senior-level signal

This shows voice as a safety-sensitive command surface, not just a pleasant chat interface.

- Human-interface realism: voice lowers friction so much that authority checks must become stronger.
- Privacy realism: always-listening systems should prefer local handling and explicit boundaries.
- Safety realism: barge-in is a control feature, not a luxury.

Skeptical reviewer questions

- What is actually built here, and what is still design? Answer: the status and boundary are stated directly inside the Voice Interface and Spoken Tool Use case.
- Which private evidence is intentionally not public? Answer: local paths, credentials, private channel details and confidential raw material are excluded.
- Why is this relevant? Answer: the case shows a reusable component for agents, tool use, memory, routing, validation or high-stakes governance.
- What would be the next stronger proof? Answer: synthetic demo, audit trace, benchmark or external review, depending on the case.

Lessons and boundaries

Wake-word/macOS app features should be described as planned or partially integrated unless retested.

- Voice commands should be treated as high-friction-to-confirm, low-friction-to-issue.
- Always-listening systems require local processing and explicit privacy boundaries.
- Barge-in is a safety feature because it lets the user stop the system quickly.
- A beautiful voice is irrelevant if authority and privacy are unclear.

Next hardening / next project step

- Retest browser/microphone E2E with a clean public demo.
- Separate safe voice tools from confirmation-required tools.
- Add transcript-level audit and user correction paths.
- Measure latency rather than relying on target budgets.
- Create a public voice simulation with transcripts instead of live microphone input.
- Separate safe tools from confirmation-required tools in the UI.
- Add an audit trail: heard phrase, parsed intent, selected tool, blocked command, confirmation and final answer.